

LOAN DEFAULT PREDICTION SYSTEM

Project Documentation

Prepared by: Ragavendra.R,

AI/ML Engineer

Document Date: May 29, 2026

Technology: Python | XGBoost | Streamlit

1. Summary

- This document describes the end-to-end design, development, and deployment of a Loan Default Risk Prediction system built for Indian lending institutions.
- The system predicts the likelihood of a borrower defaulting before loan disbursement, enabling lenders to make data-driven credit decisions and reduce non-performing assets (NPAs).
- A tuned XGBoost classifier was selected as the final model after benchmarking three algorithms, achieving a recall of 92.2% and a ROC-AUC of 96.3% on a held-out test set.
- The solution is delivered as an interactive Streamlit web application that accepts applicant details, outputs a default probability, and generates actionable recommendations.
- An estimated net financial benefit of Rs. 60.95 Crore per 10,000 applicants is projected based on average loan amounts and modelled default rates.

2. Project Overview

2.1 Problem Statement

- Loan default is a critical financial risk for banks and NBFCs operating in India's rapidly expanding credit market.
- Traditional rule-based credit scoring fails to capture complex, non-linear borrower behaviours that are predictive of default.
- The objective is to build a machine learning model that predicts default risk at the point of application using borrower profile, financial, and repayment history data.
- Binary classification target:
 - Class 0 — Non-Default: borrower successfully repays the loan.
 - Class 1 — Default: borrower fails to meet repayment obligations.

2.2 Business Objectives

- Maximise recall (sensitivity) to minimise missed defaulters — the most costly error in lending.

- Provide an interpretable risk tier (Low / Medium / High) for every application.
- Generate automated, rule-based recommendations to support loan officers.
- Deliver the model through an accessible, browser-based UI for operational teams.

2.3 Project Scope

- In scope: data cleaning, feature engineering, model training, evaluation, and Streamlit deployment.
- Out of scope: real-time credit bureau integration, production-grade REST API, and model monitoring pipeline.

3. Dataset Description

3.1 Source & Structure

- File: loan_default_risk_prediction_dataset(in).csv
- Domain: Indian lending (Retail and MSME loans)
- Total records: 51,000 raw rows reduced to 50,000 after deduplication.
- Raw columns: 43 — reduced to 40 usable features after leakage removal and redundancy cleanup.
- Class distribution: 85% non-default (Class 0), 15% default (Class 1) — moderately imbalanced.
- Train / Test split: 80% / 20% stratified — 40,000 training rows and 10,000 test rows.

3.2 Feature Categories

Category	Features Included	Count
Personal / Demographic	Age, gender, marital status, education level, residential status, location type	6
Employment & Income	Employment type, years employed, monthly income, additional income, income stability score	5
Loan Details	Loan amount, loan term, interest rate, EMI, loan purpose, loan-to-income ratio	6
Credit Profile	Credit score, existing loans, credit card usage,	4

	debt-to-income ratio	
Repayment History	Previous defaults, missed payments, on-time payment ratio, late payment count, avg delay days, repayment score	6
Financial Position	Bank balance, savings, monthly expense, savings ratio, transaction frequency	5
Time / Date Features	Loan age (days), days since last payment, days to next due, application month and day-of-week	4
Encoded (OHE) Dummies	Binary columns generated from 7 categorical variables via one-hot encoding	20
Total (after encoding)	—	50

3.3 Data Quality Issues Found and Resolved

- 7 numeric columns stored as text strings with Rs. symbols, commas, and % signs — converted to float64.
- ~5% missing values across 7 columns — filled with column median (numeric) and 'Unknown' (employment type).
- 1,000 exact duplicate rows — removed, leaving 50,000 unique records.
- Logically invalid values — 100 out-of-range ages (< 18 or > 75), 100 invalid credit scores, 100 negative incomes — replaced with median.
- Outliers in 8 financial columns (745 to 3,111 values each) — capped using IQR method ($Q1 - 1.5 \times IQR$ / $Q3 + 1.5 \times IQR$).
- 4 post-default leakage columns removed: default_notice_sent, legal_action_taken, recovery_status, settlement_status.
- 2 redundant columns dropped: annual_income ($r = 0.92$ with monthly_income) and application_year ($r = 0.96$ with loan_age_days).
- Categorical inconsistencies (e.g., 'male', 'MALE', 'Male') standardised to Title Case; abbreviations such as PG resolved to 'Post Graduate' via mapping.

4. Data Preprocessing Pipeline

4.1 Pipeline Steps (in order)

- **Step 1 - Type Conversion:** Stripped Rs./,/% from 7 financial columns; parsed all 4 date columns to datetime64.
- **Step 2 - Missing Value Imputation:** Numeric columns filled with median; employment_type filled with 'Unknown'. Result: 0 missing cells.
- **Step 3 - Duplicate Removal:** 1,000 exact duplicate rows dropped; dataset reduced to 50,000 unique records.
- **Step 4 - Invalid Value Correction:** Out-of-range values for age, credit score, income, and loan term replaced with column medians.
- **Step 5 - Outlier Capping (IQR):** Extreme values in 8 financial columns pulled to IQR boundary; data retained, not deleted.
- **Step 6 - Date Feature Engineering:** 6 temporal features extracted: loan_age_days, days_since_last_pay, days_since_next_due, application_month, application_year (later dropped), application_dayofweek.
- **Step 7 - Category Standardisation:** Title Case normalisation and manual mapping for education and employment variants.
- **Step 8 - Leakage Removal:** 4 post-outcome columns removed to prevent target leakage.
- **Step 9 - Redundancy Removal:** 2 highly correlated columns dropped ($r > 0.85$ with a retained column).
- **Step 10 - Train/Test Split:** 80/20 stratified split executed before encoding to prevent leakage. 85:15 class ratio preserved in both sets.
- **Step 11 - One-Hot Encoding:** pd.get_dummies with drop_first=True on 7 categorical columns; test set aligned to train column order.
- **Step 12 - Feature Scaling:** StandardScaler fit on train set (30 numeric columns), transform applied to both train and test.

4.2 Final Dataset Dimensions

- X_train: 40,000 rows x 50 features (scaled, encoded)
- X_test: 10,000 rows x 50 features (scaled, encoded, same column order)
- y_train: 40,000 labels — 6,000 defaults (15%)
- y_test: 10,000 labels — 1,500 defaults (15%)

5. Model Development

5.1 Algorithms Evaluated

- **Model 1 - Logistic Regression:** Baseline linear classifier with `class_weight='balanced'` and `max_iter=1000`.
 - Suitable as a benchmark; interpretable coefficients but limited capacity for non-linear patterns.
- **Model 2 - Random Forest:** Ensemble of 100 decision trees with `class_weight='balanced'` and parallel fitting.
 - Strong precision (85.3%) but critically low recall (26.6%) — unsuitable for a lending risk context.
- **Model 3 - XGBoost (Baseline):** Gradient-boosted trees with `scale_pos_weight = 5.67` to handle class imbalance.
 - Best overall performance across accuracy, recall, F1, and ROC-AUC at baseline.
- **Model 4 - XGBoost (Tuned) [FINAL]:** Grid search over 27 combinations (`max_depth`, `learning_rate`, `n_estimators`). 5-fold CV scored on ROC-AUC. Completed in 2.8 minutes (135 model fits).
 - Best parameters: `learning_rate = 0.05`, `max_depth = 4`, `n_estimators = 300`.

5.2 Model Performance Comparison

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
Logistic Regression	85.75%	51.49%	86.53%	64.56%	93.66%
Random Forest	88.30%	85.26%	26.60%	40.55%	94.59%
XGBoost (Baseline)	90.14%	61.75%	90.07%	73.26%	96.12%
XGBoost (Tuned) [*]	89.44%	59.56%	92.20%	72.37%	96.29%

- [*] Selected final model — highest recall (92.2%) and ROC-AUC (96.29%).
- XGBoost (Tuned) improved recall by +2.13 percentage points over baseline at the cost of a minor accuracy drop (-0.7%) — a deliberate, risk-aligned trade-off.
- Random Forest is disqualified despite high precision: it misses 73.4% of real defaulters, which is unacceptable for a lending risk system.

5.3 Cross-Validation Results (5-Fold Stratified, ROC-AUC)

Model	CV Mean ROC-AUC	CV Std Dev	Interpretation
Logistic Regression	0.9410	0.0035	Stable baseline — consistent across folds
Random Forest	0.9484	0.0018	Good AUC but recall failure disqualifies it
XGBoost	0.9633	0.0017	Highest AUC and lowest variance — best generalisation

- Narrow standard deviations across all models confirm no overfitting to any single train-test partition.
- Stratified folds preserve the 85:15 class ratio in each fold for a fair evaluation under class imbalance.

6. Final Model Evaluation — Tuned XGBoost

6.1 Confusion Matrix Analysis (Test Set — 10,000 Applicants)

	Predicted: No Default	Predicted: Default
Actual: No Default	7,561 True Negatives (TN)	939 False Positives (FP)
Actual: Default	117 False Negatives (FN) [!]	1,383 True Positives (TP)

- TP = 1,383: defaulters correctly identified and flagged before loan disbursement.
- FN = 117: missed defaulters — only 7.8% of all actual defaults escape detection.
- FP = 939: genuine borrowers incorrectly flagged — an 11% over-rejection rate, operationally manageable with manual review.
- TN = 7,561: clean borrowers correctly approved, preserving the customer acquisition pipeline.

6.2 Risk Tier Distribution (Test Set)

Risk Tier	Probability Range	Applicants	Actual Defaults	Default Rate
Low Risk	Probability < 31%	6,923	41	0.6%
Medium Risk	31% to 66%	1,303	215	16.5%
High Risk	Probability >= 66%	1,774	1,244	70.1%

- Low Risk tier (0.6% default rate) — safe to auto-approve with standard loan terms.
- High Risk tier (70.1% default rate) — confirms strong discriminative power; mandatory manual review or rejection.
- Medium Risk applicants (16.5% default rate) — warrant conditional approval or reduced loan amount.

6.3 Additional Metrics

- PR-AUC (Average Precision): 0.8008 — strong performance on the minority default class.
- ROC-AUC: 0.9629 — model correctly ranks a defaulter above a non-defaulter 96.3% of the time.
- The probability threshold (default: 0.5) can be adjusted to trade recall for precision based on operational requirements.

7. Feature Importance Analysis

7.1 Top 15 Features — Tuned XGBoost (by Importance Score)

Rank	Feature Name	Importance	Category
1	previous_defaults	0.1350	Repayment History
2	missed_payments_last_12_months	0.1260	Repayment History
3	employment_type_Unemployed	0.1258	Employment
4	late_payment_count	0.1030	Repayment History
5	loan_to_income_ratio	0.0948	Loan Details
6	monthly_income	0.0767	Employment & Income
7	repayment_history_score	0.0517	Repayment History
8	debt_to_income_ratio	0.0476	Credit Profile
9	credit_score	0.0332	Credit Profile
10	on_time_payment_ratio	0.0313	Repayment History
11	average_delay_days	0.0180	Repayment History
12	employment_type_Unknown	0.0107	Employment
13	loan_purpose_Medical	0.0077	Loan Details
14	savings_ratio	0.0074	Financial Position
15	residential_status_Rented	0.0065	Personal / Demographic

7.2 Key Insights

- Past payment behaviour (previous_defaults, missed_payments, late_payment_count) collectively drives ~36% of model decisions — historical repayment is the strongest predictor.
- Employment status (Unemployed flag, importance 12.6%) is the third strongest signal, directly linking job stability to default likelihood.
- Financial leverage metrics (loan_to_income_ratio rank 5, debt_to_income_ratio rank 8) confirm that borrower repayment capacity is a primary risk driver.
- Credit score, the traditional lending metric, ranks only 9th — behavioural and capacity features outperform a static score in this dataset.
- Income (monthly_income, rank 6) contributes independently beyond ratio-based features, reflecting absolute repayment ability.

8. Business Impact & Financial Estimation

8.1 Financial Impact Model (per 10,000 Applicants)

Outcome	Count	Unit Value (Assumed)	Total Impact
Defaulters correctly caught (TP)	1,383	Rs. 5,00,000 principal saved	Rs. 69.15 Crore
Defaulters missed (FN)	117	Rs. 5,00,000 principal lost	Rs. 5.85 Crore
False alarms — opportunity cost (FP)	939	Rs. 25,000 lost interest	Rs. 2.35 Crore
NET BENEFIT	—	—	Rs. 60.95 Crore

- The model generates an estimated net benefit of Rs. 60.95 Crore per 10,000 applicants assuming an average loan amount of Rs. 5 Lakh.
- Only 117 real defaulters escape detection (7.8% miss rate), representing a tightly controlled financial exposure.
- The 11% false alarm rate on genuine borrowers (939 applicants) introduces manageable overhead that can be reduced by lowering the probability threshold at the cost of slightly reduced recall.

8.2 Operational Value

- Automated pre-screening reduces manual underwriting effort for clear-cut low-risk and high-risk cases.
- Loan officers focus attention on the Medium Risk tier (1,303 applicants, 16.5% default rate) where human judgment adds the most value.
- The recommendation engine generates case-specific flags (low credit score, high DTI, prior defaults) that directly guide officer decisions.
- Risk tiering enables product differentiation:
 - Low Risk — standard interest rate, streamlined approval.
 - Medium Risk — additional verification, reduced loan amount, or co-signer requirement.
 - High Risk — mandatory manual review, collateral requirement, or rejection.

9. Application Architecture & Deployment

9.1 Streamlit Application Overview

- Framework: Streamlit (Python) with Plotly for interactive visualisations.
- Launch command: `streamlit run apps/streamlit_app.py`
- Access URL: `http://localhost:8501`
- Layout: collapsible sidebar for all 36 input fields; main panel for results and charts.

9.2 Input Sections (Sidebar)

Section	Inputs Collected
Personal Info	Age, gender, marital status, education level, residential status, location type
Employment & Income	Employment type, years employed, monthly income, additional income, income stability score, bank balance, savings, monthly expense, transaction frequency
Loan Details	Loan amount, term (months), interest rate, EMI, loan purpose
Credit Profile	Credit score, existing loans, credit card usage %, debt-to-income ratio
Repayment History	Previous defaults, missed payments (12m), on-time payment ratio, late payment count, average delay days, repayment history score
Application Date & Timing	Application date, loan age, days since last payment, days until next due

9.3 Prediction Workflow

- User fills sidebar inputs and clicks the 'Predict Default Risk' button.
- Two derived features are auto-computed: $\text{savings_ratio} = \text{savings} / \text{income}$; $\text{loan_to_income_ratio} = \text{loan} / (\text{income} \times 12)$.
- Input vector of 50 features is constructed (numeric values + one-hot encoded dummies).
- 30 numeric columns are scaled using the saved StandardScaler (fitted on training data only).
- Tuned XGBoost model outputs a default probability score (0–100%).
- Risk tier assigned: Low Risk (< 31%), Medium Risk (31–66%), High Risk ($\geq 66\%$).

- Results displayed: probability gauge chart, four KPI metric cards, risk factor warnings, and recommendations.
- Top 15 feature importances shown as an interactive horizontal bar chart via Plotly.

9.4 Saved Model Artifacts

Artifact File	Contents	Used By
xgboost_tuned.pkl	Tuned XGBoost classifier (300 trees, lr=0.05, depth=4)	Streamlit app
scaler.pkl	StandardScaler fitted on 30 numeric training columns	Streamlit app
feature_columns.pkl	Ordered list of 50 feature column names	Streamlit app
numeric_columns.pkl	List of 30 numeric column names for scaling	Streamlit app
final_pipeline.pkl	Combined pipeline: model + scaler + risk + recommendation functions	API / notebooks
xgboost_best_params.pkl	Best hyperparameters from GridSearchCV	Reference
logistic_regression.pkl	Baseline Logistic Regression model	Comparison
random_forest.pkl	Baseline Random Forest model	Comparison
xgboost_baseline.pkl	Baseline XGBoost (un-tuned)	Comparison

10.Challenges Faced and How I Handled Them

Dirty data formats — The dataset had currency values like ₹45,000 and percentages like 12.5% stored as text. I wrote custom functions to strip the symbols and convert them into proper numbers.

Mixed date formats — The date columns had three different formats mixed together (02-Aug-24, 17-08-2021, 2/21/2021). I used pandas' flexible date parser to handle all formats automatically.

Data leakage columns — I found columns like `recovery_status` and `legal_action_taken` that only get filled after a default happens. I removed them because using them would make the model cheat and fail on real applicants.

Class imbalance — Only 15% of borrowers defaulted, so the model could get 85% accuracy by predicting "no default" for everyone. I initially used SMOTE, but switched to `class_weight='balanced'` based on reviewer feedback, since synthetic data is not preferred in banking.

Streamlit prediction errors — The UI initially crashed because the saved pipeline contained function references that couldn't load outside the notebook. I fixed it by loading the model and scaler separately and keeping the helper functions inside the app file.

Feature matching in UI — User inputs had to match the model's 50 one-hot encoded columns exactly. I built a function that starts with all zeros and fills only the relevant columns, ensuring the input always matched the training format.

11. Technology Stack & Conclusion

11.1 Technology Stack

Component	Technology / Library	Notes
Language	Python 3.x	Core development language
Data Manipulation	Pandas, NumPy	Data cleaning, transformation, and analysis
Machine Learning	Scikit-learn	LR, RF, StandardScaler, GridSearchCV, metrics
Gradient Boosting	XGBoost	Final production model (scale_pos_weight = 5.67)
Interactive Charts	Plotly	Gauge chart and feature importance bar chart
Visualisation	Matplotlib, Seaborn	EDA plots and confusion matrix heatmap
Web Application	Streamlit	Single-page browser-based risk assessment UI
Model Serialisation	Joblib	.pkl artifacts for all models and preprocessors
Notebooks	Jupyter	3 notebooks: EDA, preprocessing, model training
Development IDE	Visual Studio Code	Windows 10 environment

11.2 Project File Structure

- data/raw/ — Raw CSV dataset (51,000 x 43 columns)
- data/processed/ — Cleaned, encoded, scaled train/test CSVs (4 files)
- notebooks/ — data_understanding.ipynb, preprocessing.ipynb, model_training.ipynb
- models/ — 9 serialised model and pipeline artifacts (.pkl files)
- apps/streamlit_app.py — Production Streamlit application

11.3 Conclusion

- The project delivers a production-ready loan default prediction system combining rigorous data preprocessing, multi-model benchmarking, and an interactive Streamlit deployment.
- The tuned XGBoost classifier achieves 92.2% recall and 96.3% ROC-AUC, representing state-of-the-art performance for binary credit risk classification on Indian lending data.
- Behavioural repayment features (missed payments, prior defaults, late payment count) emerge as the strongest predictors — shifting credit risk modelling beyond sole reliance on credit score.
- The three-tier risk categorisation and automated recommendation engine translate model outputs into actionable lending decisions, making the system immediately usable by loan officers.
- Future enhancements may include real-time credit bureau API integration, SHAP-based individual applicant explanations, automated threshold optimisation, and a REST API wrapper for core banking system integration.